

**RÉPUBLIQUE DU CAMEROUN**  
REPUBLIC OF CAMEROON  
*Peace – Work – Fatherland*

**UNIVERSITÉ DE DSCHANG**  
UNIVERSITY OF DSCHANG  
*Scholae Thesaurus Dschangensis Ibi Cordum*

BP 96, Dschang (Cameroun) – Tél./Fax (237) 233 45 13 81  
Website : <http://www.univ-dschang.org>  
E-mail : [udsrectorat@univ-dschang.org](mailto:udsrectorat@univ-dschang.org)



**INSTITUT UNIVERSITAIRE DE  
TECHNOLOGIE FOTSO VICTOR DE  
BANDJOUN**  
FOTSO VICTOR UNIVERSITY  
INSTITUTE OF TECHNOLOGY

**Département de Génie Informatique**  
*Department of Computer Engineering*

BP 134, Bandjoun – Tél./Fax (237) 99 31 61 30 / 70 64 23 92  
Website : <http://www.univ-dschang.org/iutfv/>  
E-mail : [iut.fotsovictor@univ-dschang.org](mailto:iut.fotsovictor@univ-dschang.org)

# SUPPORT DE COURS, TD ET TP

## ALGORITHMIQUES ET STRUCTURES DE DONNEES

© NOULAMO THIERRY  
Enseignant à l'IUT FOTSO Victor de Bandjoun  
[thierry.noulamo@gmail.com](mailto:thierry.noulamo@gmail.com)

Année académique 2018-2019

## SOMMAIRE

|                                                                                                                     |           |
|---------------------------------------------------------------------------------------------------------------------|-----------|
| <b>INTRODUCTION .....</b>                                                                                           | <b>4</b>  |
| <b>CHAPITRE 1 INTRODUCTION A L'ALGORITHMIQUE .....</b>                                                              | <b>5</b>  |
| 1. NOTION D'ALGORITHME.....                                                                                         | 5         |
| <i>Définition.</i> .....                                                                                            | 5         |
| 2. LANGAGE DE DESCRIPTION D'ALGORITHMES .....                                                                       | 5         |
| 3. CODAGE ET STRUCTURES DE CONTROLE .....                                                                           | 5         |
| a. <i>Définitions.</i> .....                                                                                        | 5         |
| b. <i>Types de base.</i> .....                                                                                      | 5         |
| 3. STRUCTURES DE CONTROLE.....                                                                                      | 6         |
| 4. FONCTIONS .....                                                                                                  | 7         |
| 5. STRUCTURES DE DONNEES .....                                                                                      | 7         |
| a) <i>Structure</i> .....                                                                                           | 7         |
| b) <i>Tableaux à une dimension (Vecteur)</i> .....                                                                  | 8         |
| c) <i>Tableau à deux dimension (Matrice)</i> .....                                                                  | 8         |
| 6. RECURSIVITE.....                                                                                                 | 8         |
| 7. LISTES .....                                                                                                     | 9         |
| 8. QUELQUES EXEMPLES D'ALGORITHMES .....                                                                            | 10        |
| <b>CHAPITRE 2 INTRODUCTION A LA COMPLEXITE.....</b>                                                                 | <b>12</b> |
| 1. INTRODUCTION .....                                                                                               | 12        |
| 2. NOTION DE COMPLEXITE .....                                                                                       | 12        |
| a) <i>Motivation et principe</i> .....                                                                              | 12        |
| b) <i>Complexités spatiale et temporelle On distingue deux types de complexités : temporelle et spatiale.</i> ..... | 13        |
| 3. EXEMPLE : 2 FONCTIONS DIFFERENTES CALCULANT LE PRODUIT DES NOMBRES D'UN TABLEAU D'ENTIERS. ....                  | 13        |
| 4. OPERATIONS EN TEMPS CONSTANT .....                                                                               | 15        |
| 5. CAS FAVORABLE, MOYEN ET DEFAVORABLE .....                                                                        | 16        |
| 6. NOTATION ASYMPTOTIQUE.....                                                                                       | 17        |
| 7. COMPLEXITE ASYMPTOTIQUE .....                                                                                    | 18        |
| 8. ALGORITHMES RECURSIFS.....                                                                                       | 18        |
| a) <i>Méthode par substitution</i> .....                                                                            | 19        |
| b) <i>exemple : calcul de factorielle</i> .....                                                                     | 19        |
| c) <i>Formulaire.</i> .....                                                                                         | 19        |
| <b>CHAPITRE 3 STRUCTURES DE DONNEES.....</b>                                                                        | <b>21</b> |
| 1. INTRODUCTION .....                                                                                               | 21        |
| 2. PILES DE DONNEES .....                                                                                           | 21        |
| a) <i>Présentation</i> .....                                                                                        | 21        |
| b) <i>Type abstrait.</i> .....                                                                                      | 21        |
| c) <i>Implémentation par tableau</i> .....                                                                          | 22        |
| d) <i>Implémentation par liste chaînée</i> .....                                                                    | 23        |
| 3. FILES DE DONNEES .....                                                                                           | 23        |
| a) <i>Présentation</i> .....                                                                                        | 23        |
| b) <i>Type abstrait.</i> .....                                                                                      | 23        |
| c) <i>Implémentation simple par tableau.</i> .....                                                                  | 24        |
| d) <i>Implémentation par liste chaînée</i> .....                                                                    | 24        |
| 4. ARBRES .....                                                                                                     | 25        |
| a) <i>Généralités</i> .....                                                                                         | 25        |
| b) <i>Arbre binaire</i> .....                                                                                       | 26        |
| c) <i>Type abstrait</i> .....                                                                                       | 26        |
| d) <i>Implémentation par pointeur</i> .....                                                                         | 27        |
| <b>CHAPITRE 4 LES FICHIERS SEQUENTIELS .....</b>                                                                    | <b>28</b> |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 1- INTRODUCTION .....                                     | 28        |
| 2- STRUCTURE FILE .....                                   | 28        |
| 3- OUVERTURE/FERMETURE .....                              | 29        |
| 4- LECTURE/ECRITURE NON-FORMATEES EN MODE CARACTERE ..... | 29        |
| 5- LECTURE/ECRITURE NON-FORMATEES EN MODE CHAINE .....    | 31        |
| <b>CHAPITRE 5 TRIS.....</b>                               | <b>32</b> |
| 1. INTRODUCTION .....                                     | 32        |
| 2. LE TRI RAPIDE .....                                    | 32        |
| a) Principe .....                                         | 32        |
| b) Implémentation .....                                   | 32        |
| 3. LE TRI FUSION .....                                    | 33        |
| a) Principe .....                                         | 33        |
| b) Implémentation .....                                   | 34        |
| 4. TRI PAR TAS : HEAPSORT .....                           | 35        |
| – Tas : arbre binaire et partiellement ordonné.....       | 35        |
| a) Principe .....                                         | 36        |
| b) Implémentation .....                                   | 36        |
| 5. COMPLEXITE DES TRIS .....                              | 37        |

# INTRODUCTION

Ce document présente les fondamentaux qui doivent guider son lecteur pour la conception, l'analyse, la production des algorithmes de qualité en utilisant des structures de données complexe. L'objectif est de développer la créativité en matière de production des structures de données spécifiques à un problème, tout en s'atardant sur la qualité de la solution proposé.

Le chapitre 1 donne une présentation générale des algorithmes et de leur place dans les systèmes informatiques modernes. Ce chapitre définit ce qu'est un algorithme et donne des exemples. Il presente aussi le formalisme qui sera utilisé tout au long de ce développement.

Au chapitre 2, la question d'efficacité des algorithmes est posée. Dans ce chapitre nous introduisons le champ de la théorie de la complexité, afin de répondre à ce type de question. Grâce aux exemples bien selectionnées, le lecteur peut assimiler en douceur les concepts.

Le chapitre 3 porte sur la présentation de quelques exemples de structure des données : les piles, les files et les arbres. L'approche se décline en la présentation de l'outil, sa description abstraite et son implémentation.

Le chapitre 5 se focalise sur les graphes comme structure de données.

Nous présentons au chapitre 5 les algorithmes de tris. La notion d'algorithme de tri de façon générale est définie, ainsi que les différents aspects utilisés lorsqu'on veut les caractériser et les comparer. Dans un deuxième temps, nous présentons en détails les algorithmes de tri les plus connus.

Une fiche de TD contenant des exercices bien selectionnés accompagne ce support.

# Chapitre 1 Introduction à l'Algorithmique

## 1. Notion d'algorithme

### Définition.

Un algorithme est une procédure de calcul bien définie qui prend en entrée un ensemble de valeurs et qui délivre en sortie un ensemble de valeurs. Un algorithme est donc une séquence d'étapes de calcul qui transforment l'entrée en sortie.

### Exemple

**Problème** : Trier une suite de nombres entiers dans l'ordre croissant.

**Entrée** : Suite de  $n$  nombres entiers ( $a_1, a_2, \dots, a_n$ )

**Sortie** : Une permutation de la suite donnée en entrée ( $a'_1, a'_2, \dots, a'_n$ ) telle que  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ .

On appelle Instance du problème une valeur particulière de l'ensemble des valeurs données en entrée.

**Exemple** La valeur (6,9,2,4) est une instance du problème.

Un algorithme est correct si pour toute instance du problème il se **termine** et produit une sortie **correcte**.

Une structure de données est un moyen de stocker et d'organiser des données pour faciliter leur stockage, leur utilisation et leur modification.

L'efficacité d'un algorithme est mesuré par son coût (complexité) en temps et en mémoire.

## 2. Langage de description d'algorithmes

Il est nécessaire de disposer d'un langage qui soit non lié à l'implémentation. Ceci permet une description plus précise des structures de données ainsi qu'une rédaction de l'algorithme plus souple et plus "lisible". Le langage **EXALGO** est un exemple de ce qui peut être utilisé et qui sera utilisé dans ce cours. Il est composé de chaînes de caractères alphanumériques, de signes opératoires (+, -, \*, /, <, <=, >=, >, <>, ==, !=, ou, non, et), de mot-clés réservés, et de signes de ponctuation : "=", ":", "(", ")", "début", "fin", "//. Les balises début et fin peuvent être remplacés par { et }.

## 3. Codage et structures de contrôle

### a. Définitions

**Définition 1** Un type abstrait est un triplet composé : d'un nom, d'un ensemble de valeurs, d'un ensemble d'opérations définies sur ces valeurs. Les types abstraits de bases de l'algorithmique sont : entier, caractères, booléen, réel que l'on écrit respectivement en EXALGO entier, car, booléen, réel

**Définition 2** Une variable est un triplet composé d'un type (déjà défini), d'un nom (a priori toute chaîne alphanumérique), d'une valeur.

On écrit en EXALGO **var NomDeVariable: Type;**

Type est à prendre pour l'instant dans l'ensemble {entier, car, booléen, réel}

**Définition 3** Les Expressions sont constituées à l'aide de variables déjà déclarées, de valeurs, de parenthèses et d'opérateurs du (des) type(s) des variables concernées.

**Définition 4** L'affectation est l'instruction qui permet de stocker une valeur dans une variable. On écrit **NomDeVariable=ExpressionDuTypeDeLaVariable;**

**NB : Toute variable doit être déclarée et recevoir une valeur initiale.**

### b. Types de base

#### Booléens

Une variable de type booléen prend comme valeur VRAI ou FAUX. Les opérations usuelles sont ET, OU et NON qui sont données dans les tables qui suivent.

#### Entiers

Une variable de type entier peut prendre comme valeur l'ensemble des nombres entiers signés. Les opérations associées sont les opérations usuelles +,-,\*,/.

### **Réels**

Une variable de type réel peut prendre comme valeur l'ensemble des nombres réels. Les opérations associées sont les opérations usuelles +,-,\*,/.

### **Caractères**

Une variable de type car peut prendre comme valeur l'ensemble des caractères imprimables.

On notera les valeurs entre guillemets. On considère souvent que les caractères sont ordonnés dans l'ordre alphabétique.

### **Opérateurs de Comparaison**

Les opérateurs  $=$ ,  $\geq$  permettent de comparer les valeurs de type entier, réel et caractère. Le résultat de cette comparaison est une valeur booléenne.

## **3. Structures de contrôle**

Il y a trois structures principale de contrôle qui permettent de construire des algorithmes

### **Bloc d'instruction**

```
début
    instruction1
    instruction2
    .....
fin
```

### **Alternative**

#### **Alternative simple:**

```
si ExpressionBooléenne alors
    BlocInstruction1
sinon
    BlocInstruction2
finsi;
```

#### **Alternative multiple:**

```
selon que
    cas cas1 : BlocInstruction1
    cas cas2 : BlocInstruction2
    .....
    autrement : BlocInstruction
finselorque
```

### **Répétition**

L'instruction exit permet d'arrêter la répétition.

- le bloc d'instruction peut ne pas être exécuté  
tant que *ExpressionBooléenne* faire  
    *BlocInstruction*  
fintantque;
- le bloc d'instruction est exécuté au moins une fois  
répéter  
    *BlocInstruction*  
    jusqu'à *ExpressionBooléenne*  
finrépéter;
- bloc d'instruction peut ne pas être exécuté et il y a une variable indicatrice:  
pour *VariableIndicatrice* allant de *ValeurInitiale* à *ValeurFinale* par pas de *ValeurPas* faire  
    *BlocInstruction*

*finpour;*

#### 4. Fonctions

Une fonction est une section d'algorithme qui a un objectif bien défini et un nom. En général, elle communique avec l'extérieur par le biais de paramètres typés. Elle possède des variables locales qui ne sont pas visibles à l'extérieur de la fonction. Ces variables peuvent être des fonctions. Une fonction retourne une valeur par l'instruction simple *retourne(Expression)*.

L'expression peut être

- vide, tout s'est bien passé mais il n'y a pas de résultat à retourner : *retourne()*
- sans résultat, il est impossible de retourner un résultat suite à un cas de figure de l'instance : *retourne(NUL)*

##### Syntaxe

```
fonction NomDeFonction (ListeParamètres):TypeRésultat;  
    //déclarations des variables ou fonctions locales autres que les paramètres  
    début  
        // partie instruction qui contient l'appel à retourner  
    fin  
finFonction
```

Les paramètres sont passés **par référence ref**, on écrit **ref ListeVariable:NomDeType**

la fonction travaille directement dans la variable passée en paramètre,

**par valeur val**, on écrit **val ListeVariable:NomDeType**

la fonction travaille sur une copie de la variable passée en paramètre. Le type du résultat est vide si la fonction ne renvoie pas de résultat.

Une fonction s'utilise en écrivant **NomDeFonction(ListeInstanceParamètres)** dans le calcul d'une expression si la fonction retourne une valeur, comme une instruction simple si elle ne retourne pas de valeur.

#### 5. Structures de données

##### a) Structure

Une structure. est un mécanisme permet de définir de nouveaux types plus complexes que les types de base. En EXALGO, on écrit

```
nom_du_type=structure  
    nom_champs_1:type1;  
    nom_champs_2:type2;  
    .....  
    nom_champs_k:typek;  
finstructure.
```

Ceci signifie que lorsqu'une variable est déclarée de ce type, elle référence k variables en même temps.

Soit V une variable dont le type est une structure, on désigne un des champs par V. suivi du nom du champs.

**Exemple 3.4** Une date de naissance est un exemple de structure. On peut écrire :

```
dateDeNaissance=structure  
    jourDeNaissance:entier;  
    moisDeNaissance:entier;  
    annéeDeNaissance:entier;  
finstructure.
```

On peut définir une structure composée du sexe et de la date de naissance par :

```
individu=structure
    sexe:booléen ;
    date:dateDeNaissance;
finstructure.
```

Soit la déclaration

var I:individu alors **I.sexe** sera un booléen et **I.date.jourDeNaissance** sera un entier.

Ainsi les instructions suivantes ont un sens:

```
I.date.jour=12;
I.sexe=false;
```

### b) Tableaux à une dimension (Vecteur)

Un tableau est une table d'association à clé unique telle que le nombre d'éléments de la table (dimension ou taille) est constant, l'accès aux éléments s'effectue directement par la clé, les valeurs minimum et maximum des clés sont des constantes. On écrit en EXALGO :

**nom\_tableau=tableau[min\_indice..max\_indice] de type\_predefini;**

ce qui signifie que :

- les éléments ont pour type le type\_prédéfini
- les indices des éléments vont de min\_indice à max\_indice, avec min\_indice<max\_indice

#### Exemple Fonction d'initialisation d'un vecteur

```
fonction init(ref T:tableau[min_indice..max_indice] d'éléments; val
valeurInitiale:élément):vide;
    var i:entier;
    début
        pour i allant de min_indice à max_indice faire
            T[i]=valeurInitiale ;
        finpour
    fin
finfonction
```

### c) Tableau à deux dimension (Matrice)

Une matrice M de dimension nxm est un tableau de dimension n dont chaque élément est un tableau de dimension m. On peut donc déclarer la matrice sous la forme suivante :

#### Exemple Fonction d'initialisation d'une matrice

```
fonction initMatrice(ref M:tableau[1..n] de tableau [1..m] d'éléments; val
valeurInitiale:élément):vide;
    var i,j:entier;
    début
        pour i allant de 1 à n faire pour j allant de 1 à m faire
            M[i][j]=valeurInitiale ;
        finpour
    retourner() ;
finfonction
```

## 6. Recursivité

La récursivité consiste à remplacer une boucle par un appel à la fonction elle-même. Considérons la suite factorielle, elle est définie par :

```
0!=1
n!=n(n-1)!
```

La fonction peut s'écrire simplement



```

fonction factorielle(val n:entier):entier;
    début
        si (n==0)
            retourne(1) ;
        sinon
            retourne(factorielle(n-1)*n) ;
        finsi
    fin
finfonction;

```

## 7. Listes

Une liste est une table d'association à clé unique telle que le nombre d'éléments de la table (dimension ou taille) est variable, l'accès aux éléments s'effectue indirectement par le contenu de la clé qui le localise appelée pointeur.

Un élément d'une liste est l'ensemble (ou structure) formé :

- d'une donnée ou information,
- d'un pointeur nommé Suivant indiquant la position de l'élément le suivant dans la liste.

A chaque élément est associée une adresse mémoire.

Les listes chaînées font appel à la notion de variable dynamique.

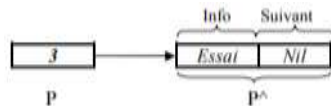
Une variable dynamique:

- est déclarée au début de l'exécution d'un programme,
- elle y est créée, c'est-à-dire qu'on lui alloue un espace à occuper à une adresse de la mémoire,
- elle peut y être détruite, c'est-à-dire que l'espace mémoire qu'elle occupait est libéré,
- l'accès à la valeur se fait à l'aide d'un pointeur.

Un pointeur est une variable dont la valeur est une adresse mémoire. Un pointeur, noté P, pointe sur une variable dynamique notée P<sup>^</sup>.

Le type de base est le type de la variable pointée. Le type du pointeur est l'ensemble des adresses des variables pointées du type de base. Il est représenté par le symbole  $\wedge$  suivi de l'identificateur du type de base.

Exemple:



La variable pointeur P pointe sur l'espace mémoire P<sup>^</sup> d'adresse 3. Cette cellule mémoire contient la valeur "Essai" dans le champ Info et la valeur spéciale Nil dans le champ Suivant. Ce champ servira à indiquer quel est l'élément suivant lorsque la cellule fera partie d'une liste. La valeur Nil indique qu'il n'y a pas d'élément suivant. P<sup>^</sup> est l'objet dont l'adresse est rangée dans P.

Les listes chaînées entraînent l'utilisation de procédures d'allocation et de libération dynamiques de la mémoire. Ces procédures sont les suivantes:

- Allouer(P) : réserve un espace mémoire P<sup>^</sup> et donne pour valeur à P l'adresse de cet espace mémoire. On alloue un espace mémoire pour un élément sur lequel pointe P.
- Désallouer(P) : libère l'espace mémoire qui était occupé par l'élément à supprimer P<sup>^</sup> sur lequel pointe P.

Pour définir les variables utilisées dans l'exemple ci-dessus, il faut :  
définir le type des éléments de liste :

```

Type Cellule= Structure
    Info : TypeElement ;
    Suivant : Liste

```

finStructure

- définir le type du pointeur : Type Liste = ^Cellule
- déclarer une variable pointeur : Var P : Liste
- allouer une cellule mémoire qui réserve un espace en mémoire et donne à P la valeur de l'adresse de l'espace mémoire P<sup>^</sup> : Allouer(P)
- affecter des valeurs à l'espace mémoire P<sup>^</sup>: P<sup>^</sup>.Info="Essai" ; P<sup>^</sup>.Suivant=Nil

Quand P = Nil alors P ne pointe sur rien.

### Listes chaînées simples

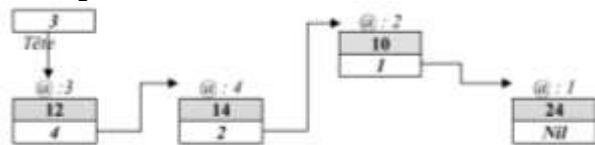
Une liste chaînée simple est composée :

- d'un ensemble d'éléments tel que chacun :
  - est rangé en mémoire à une certaine adresse,
  - contient une donnée (Info),
  - contient un pointeur, souvent nommé Suivant, qui contient l'adresse de l'élément suivant dans la liste,
- d'une variable, appelée Tête, contenant l'adresse du premier élément de la liste chaînée.

Le pointeur du dernier élément contient la valeur Nil. Dans le cas d'une liste vide le pointeur de la tête contient la valeur Nil. Une liste est définie par l'adresse de son premier élément.

Avant d'écrire des algorithmes manipulant une liste chaînée, il est utile de montrer un schéma représentant graphiquement l'organisation des éléments de la liste chaînée.

**Exemple:.**



Si P a pour valeur 3

P<sup>^</sup>.Info a pour valeur 12

P<sup>^</sup>.Suivant a pour valeur 4

Si P a pour valeur 2

P<sup>^</sup>.Info a pour valeur 10

P<sup>^</sup>.Suivant a pour valeur 1

### Déclarations des types pour la liste :

Type Liste = ^Element

Type Element = Structure

Info : TypeElement

Suivant : Liste

finStructure

### En exercice : Traitements de base d'utilisation d'une liste chaînée simple

- Création d'une liste „
- Parcours d'une liste „
- Accès à un élément d'une liste „
- Mises à jour d'une liste

Autres types de liste :

- Listes chaînées circulaires
- Listes doublement chaînées

## 8. Quelques exemples d'algorithmes

- Recherche d'un élément : version itérative et recursive
- Tri non récursif (Insertion, selection, tri bulle) : version itérative et recursive
- Fusion de tableau trié : version itérative et recursive
- Trouver un élément dans un tableau ordonné : version itérative et recursive
- implémentation des polynômes



# Chapitre 2 Introduction à la Complexité

## 1. Introduction

Comme on l'a vu lors de l'implémentation des types abstraits, il est souvent possible d'implémenter plusieurs solutions à un problème donné. La question se pose alors de savoir quel algorithme est le plus efficace. Et pour effectuer cette comparaison, il est nécessaire de déterminer sur quels critères elle doit être effectuée. Le but de cette section est d'introduire le champ de la théorie de la complexité, dont l'un des buts est de répondre à ce type de question.

## 2. Notion de complexité

### a) Motivation et principe

**Théorie de la complexité** : théorie développée dans le but de comparer efficacement différents algorithmes entre eux, Afin de savoir lequel était le meilleur pour un problème donné. Jusqu'aux années 70, on caractérisait les algorithmes de manière empirique, en considérant :

- Le temps d'exécution ;
- L'espace mémoire utilisé ;
- Les conditions d'exécution :
  - Taille des données
  - Type d'ordinateur
  - Système d'exploitation
  - Langage de programmation
  - Etc.

**exemple** : Pour le problème consistant à calculer  $12^{34}$ , sur un P4 à 3 GHz, avec un OS Windows XP et une implémentation en langage C, supposons que l'algorithme *A* met 1,2 secondes et utilise 13 Mo de mémoire, alors que l'algorithme *B* met 1,3 secondes et utilise 0,8 Mo. Alors on peut dire que *A* est plus rapide que *B*, mais que *B* est moins gourmand en mémoire.

Le problème de cette démarche est qu'elle ne permet pas de comparer les algorithmes de manière objective, car ils ne sont pas les seuls facteurs intervenant dans la performance de résolution du problème : d'autres facteurs extérieurs interviennent également (matériel, logiciel...). La solution retenue est de se détacher de ses facteurs extérieurs, et donc de l'implémentation de l'algorithme. Pour cela, nous allons utiliser une approche plus théorique, reposant sur deux notions : les opérations élémentaires et les positions mémoire.

**Opération élémentaire** : Une opération atomique, correspondant à une instruction assembleur.

La notion d'opération élémentaire va être utilisée pour représenter la consommation que l'algorithme analysé effectue en termes de temps de calcul. On dira par exemple qu'il a besoin d'effectuer  $x$  opérations élémentaires pour résoudre le problème traité.

**Position mémoire** : unité de mémoire élémentaire, correspondant généralement à un octet.

La notion de position mémoire est une abstraction de l'occupation qu'un algorithme a de la mémoire. Elle va nous permettre d'évaluer cette consommation de façon indépendante de certaines conditions d'exécution mentionnées précédemment, en considérant le nombre de positions mémoires qu'il utilise.

Ces deux notions nous permettent d'exprimer la complexité d'un algorithme en fonction de la taille des données qu'il traite.

**Taille des données d'un problème** : entier(s) représentant la grandeur des paramètres reçus par l'algorithme devant résoudre le problème.

Notez bien que la taille peut n'être représentée par un seul entier, mais aussi par plusieurs entiers distincts. Leur nombre et leur signification exacte dépendent fortement de la nature du problème étudié :

- Nombre d'éléments à traiter ;
- Grandeur des éléments à traiter ;
- etc.

exemples :

- Tri d'un tableau de taille  $N$  : la taille des données est  $N$ , le nombre d'éléments du tableau.
- Calcul de complément de  $p$  dans  $n$  : la taille des données est le couple  $n, p$ . L'idée est que si la taille est petite, le problème sera vraisemblablement plus facile à résoudre que si elle est énorme. On veut généralement savoir comment la complexité de l'algorithme évolue quand on fait grandir la taille du problème.

**Complexité d'un algorithme : nombre d'opérations élémentaires ou de positions mémoire dont l'algorithme a besoin pour résoudre un problème d'une certaine taille.**

Formellement, l'expression d'une complexité prend la forme d'une fonction mathématique de la taille des données.

exemples :

- Si le tri du tableau a une complexité  $f(N) = an + b$ , on dira que le tri possède une complexité linéaire.
- Si un autre algorithme de tri possède une complexité  $f(N) = n^2$ , alors on considèrera que ce tri a une complexité quadratique.
- La complexité du second tri est supérieure à celle du premier.

**b) Complexités spatiale et temporelle On distingue deux types de complexités : temporelle et spatiale.**

**Complexité temporelle : nombre total d'opérations élémentaires pour exécuter l'algorithme.**

La complexité temporelle correspond donc au décompte de toutes les opérations élémentaires que l'algorithme a besoin d'effectuer pour résoudre le problème.

**Complexité spatiale : nombre maximal de positions mémoire utilisées au cours de l'exécution.**

À la différence des opérations élémentaire, qui s'enchaînent de façon séquentielle, l'occupation mémoire est une quantité qui évolue au cours du temps : en fonction de l'état de la pile (appels de fonction) et du segment de données (allocation dynamique) l'algorithme peut augmenter et diminuer le nombre de positions mémoire qu'il utilise.

La complexité spatiale correspond à l'occupation maximale atteinte par l'algorithme au cours de son exécution. Attention donc à ne pas considérer le total de toutes les positions mémoires occupées au cours de l'exécution, ou bien juste l'occupation mémoire observée juste avant que l'algorithme se termine.

**3. exemple : 2 fonctions différentes calculant le produit des nombres d'un tableau d'entiers.**

```
fonction produit1(var tab : tableau(1..N) de entier) : entier ;
1 {   var i: entier;
2     var resultat : entier;
3     Resultat:=1;
4     pour i allant de 0 à N par pas de 1 faire
5         resultat = resultat*tab[i];
6     retourne resultat;
```

```
}  
fonction produit2(var tab : tableau(1..N) de entier) : entier ;  
1 {   var i: entier;  
2     i=0 ;  
3     var resultat : entier;  
4     Resultat=1;  
5     tant que resultat !=0 et i<N faire  
6     {  
7         resultat = resultat*tab[i];  
8         i=i+1;  
9     }  
10    retourne resultat;  
}
```

- Taille des données :  $N$
  - **Complexités spatiales :**
    - Supposons qu'un entier occupe  $m$  positions mémoire.
    - Alors pour les deux fonctions, on a :
      - Tableau de  $N$  entiers.
      - Variables  $i$  et  $resultat$ .
      - **total :  $S1(N) = S2(N) = m(2 + N)$**
  - **Complexités temporelles :**
  - On suppose que chaque opération/instruction correspond à un certain nombre d'opérations élémentaires :
    - multiplication : a.
    - addition : b.
    - comparaison : c.
    - affectation : d.
    - instruction return : e.
    - et logique : f.
  - **Première fonction :**
    - 3 : affectation de  $resultat$  (d).
    - 4 : affectation de  $i$  (d).
    - Dans le for :**
      - 4 : comparaison « allant de 1 à  $N$  » (c) et incrémentation de  $i$  (une addition(b) et une affectation (d)).
      - 5 : opération \* (a) et affectation à  $resultat$  (d).
      - Nombre de répétitions du for :  $N$
      - 4 : comparaison du pour à la sortie de la boucle (c).
      - Retour de la valeur  $resultat$  (e).
- On a :  $d + d + N(c + b + d + a + d) + c + e = 2d + N(a + b + c + 2d) + c + e$
- Total :  $T1(N) = Na + Nb + (N + 1)c + (N + 1)2d + e$**
- **Seconde fonction :**
  - 2 : affectation de  $i$  (d).
  - 4 : affectation de  $resultat$  (d).
  - Dans le while :**
    - 5 : comparaisons  $resultat != 0$  (c) et  $i < N$  (c), opération &&(f).
    - 7 : opération \* (a) et affectation à  $resultat$  (d).

- 8 : incrémentation de i (une addition (b) et une affectation(d)).

**Nombre de répétitions du tant que :**

- M si le tableau contient un zéro en  $(M - 1)^{\text{ème}}$  position.
- N si le tableau ne contient pas de zéro.
- 5 : comparaisons du while à la sortie de la boucle :
- 5 comparaison resultat!=0 (c) si le tableau contient un zéro.
- 5 comparaisons (c) et un « et » (f) si le tableau ne contient pas de zéro.
- Retour de la valeur resultat (e).

on a :

- $d + d + M(c + f + c + a + d + b + d) + c + e = 2d + M(a + b + 2c + 2d + f) + c + e$
- $d + d + N(c + f + c + a + d + b + d) + c + f + c + e = 2d + N(a + b + 2c + 2d + f) + 2c + f + e$

**Total :**

$$T_2(N) = \begin{cases} (Ma + Mb + (2M + 1)c + (M + 1)2d + e + Mf) & \text{si } \exists M: \text{tab}[M - 1] = 0 \\ Na + Nb + (2N + 1)c + (N + 1)2d + e + (N + 1)f & \text{sinon} \end{cases}$$

**Remarque :** il existe un lien entre les complexités spatiale et temporelle d'un algorithme. D'abord, sur une machine mono-processeur, la complexité spatiale ne peut pas dépasser la complexité temporelle, puisqu'il faut effectuer au moins une opération pour initialiser une position mémoire.

#### 4. Opérations en temps constant

Nous avons dans le paragraphe précédent calculé la complexité spatiale et temporelle d'un algorithme. Cependant, la comparaison des complexités temporelles reste difficile, car on dépend encore trop des machines utilisées. En effet, le nombre d'opération élémentaires associé à une instruction/opération est dépendant de la machine utilisée. Par exemple, une multiplication peut correspondre à une seule opération élémentaire sur une machine, et à plusieurs sur une autre machine (par exemple une séquence d'additions).

De la même façon, pour la complexité spatiale, la représentation des données en mémoire varie suivant le langage et la machine. Par exemple, le nombre d'octets utilisés pour représenter un entier int en langage C n'est pas standard.

Déterminer certaines valeurs pose donc problème :

- Nombre d'opérations élémentaires associé à une instruction/opération ;
- Nombre d'emplacements mémoire associé à un type de donnée.

On n'a pas la possibilité de les connaître en général, i.e. de déterminer les valeurs des  $a, \dots, f$  utilisés dans l'exemple précédent.

Une solution consiste à réaliser les simplifications, suivantes :

- Une instruction/opération correspond à un nombre constant  $c$  d'opérations élémentaires.
- Une variable de type simple (entier, booléen, réel, etc.) occupe un nombre constant  $c$  d'emplacements mémoire.
- Un ensemble de variables de type simple occupe un nombre de positions mémoire dépendant de la taille des données :
  - $\text{tab}[N]$  occupe  $Nc$  positions.
  - $\text{mat}[N][M]$  en occupe  $N \times M c$ .
  - Etc.

On parle alors d'opérations en temps constant et d'emplacements en espace constant. Comme nous le verrons plus tard, cette simplification ne porte pas à conséquence dans le cadre de la comparaison d'algorithmes.

exemple : supposons que toutes les instructions/opérations nécessitent  $c$  opérations élémentaires. Les complexités des fonctions de l'exemple deviennent alors respectivement :

$$T_1(N) = Na + Nb + (N + 1)c + (N + 1)2d + e = (5N + 4)c.$$

$$T_2(N) = \begin{cases} (Mc + Mc + (2M + 1)c + (M + 1)2c + c + Mc) = (7M + 4)c \\ Nc + Nc + (2N + 1)c + (N + 1)2c + c + (N + 1)c = (7N + 6)c \end{cases}$$

On peut alors facilement comparer les complexités spatiales des algorithmes :

□ si  $M > \frac{5}{7}N$ , c'est-à-dire : s'il n'y a pas de zéro dans le tableau, ou bien s'il y a un zéro dans les deux derniers septièmes du tableau, alors produit1 est plus efficace que produit2, car  $T_1(N) < T_2(N)$ .

□ si  $M < \frac{5}{7}N$ , c'est-à-dire s'il y a un zéro dans les cinq premiers septièmes du tableau, alors produit2 est plus efficace que produit1.

## 5. Cas favorable, moyen et défavorable

Dans l'exemple précédent, la fonction produit2 possède une complexité temporelle qui varie en fonction des données d'entrée (i.e. le tableau à traiter). En effet, suivant la présence et la position du zéro, la fonction peut s'arrêter avant d'avoir atteint la fin du tableau.

En fonction des données traitées, on peut distinguer trois situations :

- Meilleur des cas : le zéro est situé au début du tableau → la fonction s'arrête tout de suite.
- Pire des cas : il n'y a pas de zéro dans le tableau, ou bien à la fin → la fonction parcourt le tableau jusqu'à la fin.
- Les autres cas : un zéro est présent dans le tableau (ailleurs qu'au début ou à la fin) → la fonction n'ira pas jusqu'au bout du tableau.

Sur ce principe, un algorithme pourra caractérisé en considérant sa complexité dans trois situations différentes, jugées particulièrement intéressantes : meilleur des cas, pire des cas, et en moyenne.

**Complexité dans le meilleur des cas :** complexité de l'algorithme quand les données traitées aboutissent à la complexité minimale. On parle aussi de cas favorable.

On rencontre rarement cette complexité, car le cas le plus favorable rend souvent trivial le problème traité, et n'est pas très discriminant (i.e. il ne permet pas de comparer les algorithmes de façon pertinente). De plus, ce cas est en général rare parmi l'ensemble des entrées possibles du domaine.

**Complexité dans le pire des cas :** complexité de l'algorithme quand les données traitées aboutissent à la complexité maximale. On parle aussi de cas défavorable.

Il s'agit de la complexité que l'on rencontre le plus souvent dans la littérature quand on veut décrire un algorithme. En effet, même si le cas défavorable est susceptible d'être rare, à l'instar du cas favorable, en revanche il est plus discriminant. Il est aussi plus pertinent d'obtenir une borne supérieure du temps d'exécution ou de l'occupation mémoire. La complexité au pire des cas est par définition le maximum des coûts, sur toutes les données de taille  $n$

$$C(n) = \max_{|x|=n} c(x)$$



**Complexité moyenne :** complexité de l'algorithme moyennée sur l'ensemble de toutes les données possibles.

On peut considérer que c'est la complexité la plus représentative de l'algorithme.

Cependant, elle est généralement difficile à calculer, et on ne la rencontre donc pas fréquemment.

$$\gamma(n) = \sum_{|x|=n} p(x)c(x)$$

Sa formule générale est : , où  $p(x)$  est la probabilité de la donnée  $x$  de taille  $n$ .

Le plus souvent, on suppose que la distribution est uniforme, c'est-à-dire que  $p(x) = 1/T(N)$ , où  $T(n)$  est le nombre de données de taille  $n$ . Alors, l'expression

$$\gamma(n) = \frac{1}{T(n)} \sum_{|x|=n} c(x)$$

du coût moyen prend la forme :

**exemple : on considère la fonction produit2**

- Pire des cas :  $T_2(N) = (7N + 6)c$
- Meilleur des cas ou cas favorable :  $T_2(N) = (7 \times 1 + 4)c = 11c$
- Cas moyen :
  - $N + 1$  cas sont possibles :
    - Soit pas de zéro.
    - Soit zéro dans une position parmi  $0; \dots; (N - 1)$ .
  - Si on suppose que ces  $N + 1$  cas sont équiprobables, alors :

$$T_2(N) = \frac{(7N + 6)c + \sum_{i=1}^N (7i + 4)c}{N + 1}$$

## 6. Notation asymptotique

On évalue l'efficacité d'un algorithme en donnant l'ordre de grandeur du nombre d'opérations qu'il effectue lorsque la taille du problème qu'il résout augmente. On parle ainsi d'algorithme linéaire, quadratique, logarithmique, etc. Les notations de Landau sont un moyen commode d'exprimer cet ordre de grandeur. Trois situations sont décrites par ces notations. La plus fréquente, la notation  $O$  introduite ci-dessous, donne une majoration de l'ordre de grandeur; la notation  $\Omega$  en donne une minoration, et la notation  $\Theta$  deux bornes sur l'ordre de grandeur.

Rappelons d'abord formellement ces notations, avant de montrer comment elles peuvent être utilisées dans le cadre de la complexité algorithme. Soient  $f$  et  $g$  deux fonctions définies de  $\mathbb{N}$  dans  $\mathbb{N}$  :

| notation          | définition                                                                        | signification                                                                                        |
|-------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| $f \in O(g)$      | $\exists a > 0, \exists x_0 > 0$<br>$\forall x \geq x_0: f(x) \leq a \times g(x)$ | $f$ croît moins vite (ou aussi vite) que $g$<br>( $g$ est une borne supérieure asymptotique de $f$ ) |
| $f \in \Omega(g)$ | $\exists a > 0, \exists x_0 > 0$<br>$\forall x \geq x_0: f(x) \geq a \times g(x)$ | $f$ croît plus vite (ou aussi vite) que $g$<br>( $g$ est une borne inférieure asymptotique de $f$ )  |
| $f \in \Theta(g)$ | $f \in O(g) \wedge f \in \Omega(g)$                                               | $f$ et $g$ croissent à la même vitesse<br>( $g$ est une borne approchée asymptotique de $f$ )        |

Dans ce cours, on utilisera essentiellement la notation en grand  $O$ , qui permet de majorer le comportement d'une fonction. Bien sûr, on veut obtenir la borne supérieure de degré le plus petit possible. La notation en  $\Theta$  sera également utilisée, plus rarement, quand il est possible de borner la complexité de l'algorithme de manière plus précise.

On distingue habituellement différentes familles d'algorithmes en fonction de leur borne supérieure :

- Complexité constante :  $O(1)$ .

- ☐ Complexité logarithmique :  $O(\log x)$ .
- ☐ Complexité log-linéaire :  $O(x \log x)$ .
- ☐ Complexité linéaire :  $O(n)$ .
- ☐ Complexité quadratique :  $O(x^2)$ , cubique  $O(n^3)$ , ou plus généralement : polynômiale  $O(x^k)$ .
- ☐ Complexité exponentielle  $O(x^k)$ .

**Remarque :** en informatique, quand la base d'un logarithme n'est pas précisée, c'est qu'il s'agit d'un logarithme de base 2.

Pour simplifier le calcul de complexité, on utilise les abus de notation suivants :

- ☐  $f \in O(g)$  sera noté  $f = O(g)$
- ☐ Dans une formule,  $O(g)$  représente une fonction anonyme appartenant à  $O(g)$
- ☐  $O(x^0)$  sera noté  $O(1)$

### Exemple

Dans  $2x^2 + 3x + 1 = 2x^2 + o(x)$ ,  $o(x)$  représente toute fonction  $f$  appartenant à  $o(x)$  tout en vérifiant l'égalité. Ici il s'agit de  $f(x) = 3x + 1$ .

Les propriétés suivantes seront utilisées lors des calculs de complexité :

- ☐ Soit  $p(x)$  un polynôme de degré  $d$ , alors on a :  $p(x) = \Theta(x^d)$
- ☐  $O(f) + O(g) = O(f + g) = O(\max(f, g))$  (distributivité pour l'addition)
- ☐  $O(f) O(g) = O(fg)$  (distributivité pour le produit)
- ☐  $\forall a, b > 0: O(\log_a n) = O(\log_b n)$

En pratique, pour obtenir la borne asymptotique supérieure une fois qu'on a exprimé  $f$  sous la forme d'un polynôme, il suffit de ne conserver que le terme de degré le plus élevé, et d'ignorer les coefficients constants.

Exemple :  $2x^2 + 3x + 1 = o(2x^2) + o(3x) + o(1) = o(2x^2) = o(x^2)$

## 7. Complexité asymptotique

L'intérêt de la notation asymptotique est double. D'une part, les calculs de complexité sont simplifiés. D'autre part, l'information utile est préservée (on obtient l'ordre de grandeur). exemple : on calcule les complexités asymptotiques des fonctions produit1 et produit2 dans le pire des cas. Toutes les opérations élémentaires ont une complexité asymptotique constante :  $O(1)$ .

- Complexités temporelles :
  - Première fonction :  
 $T_1(N) = O((5N + 4)c) = O(5Nc) = O(N)$
  - Seconde fonction :  
 $T_2(N) = O((7N + 6)c) = O(7Nc) = O(N)$

Si on compare les complexités temporelles dans le pire des cas, les deux fonctions ont la même complexité asymptotique. Pourtant, on sait (d'après nos calculs précédents) que les deux fonctions n'ont pas la même complexité dans le pire des cas :  $T_1(N) = 4 + 5N$  et  $T_2(N) = 6 + 7N$ .

**Le fait que les deux fonctions aient la même complexité asymptotique signifie que lorsque  $N$  tend vers l'infini, la différence entre  $T_1$  et  $T_2$  est négligeable.**

## 8. Algorithmes récursifs

Le calcul de complexité pour des algorithmes récursifs est plus délicat qu'il ne l'était pour les algorithmes itératifs. Dans cette sous-section, nous décrivons une méthode permettant de résoudre la relation de récurrence, et un formulaire utilisable en examen est donné pour tirer parti des résultats présentés ici. Il s'agit de la méthode par substitution.

**a) Méthode par substitution**

Le principe de cette méthode est le suivant :

- Dans l'équation définissant la complexité de la fonction, on remplace les  $O(n)$  par des polynômes en  $O(n)$ , en utilisant des constantes arbitraires.
- Dans la définition de  $T(n)$  obtenue, on substitue récursivement les  $T(h(n))$  par leur définition, jusqu'à voir apparaître une relation entre  $T(n)$  et  $T(h^i(n))$ .
- On prouve cette relation.
- On en déduit une expression non-récursive de  $T(n)$ .

**b) exemple : calcul de factorielle**

1. On remplace dans  $T(n)$ :

o Cas d'arrêt :  $T(1) = O(1) = a$

o Cas général :  $T(n) = O(1) + T(n-1) = b + T(n-1)$

2. On tente de trouver une relation à démontrer :

$$T(n) = b + T(n-1) \quad (1)$$

$$T(n-1) = b + T(n-2) \quad (2)$$

On remplace (2) dans (1) :

$$T(n) = b + (b + T(n-2)) = 2b + T(n-2)$$

**On recommence :**

$$T(n) = 2b + (b + T(n-3)) = 3b + T(n-3) \quad (4)$$

$$T(n) = 3b + (b + T(n-4)) = 4b + T(n-4) \quad (5)$$

etc.

On peut faire l'hypothèse qu'on s'intéresse à la propriété  $P_i$  telle que :  $T(n) = ib + T(n-i)$  pour  $1 < i < n$ .

3. Prouvons  $P_i$  :

o Cas de base :  $i = 1$

Le cas de base correspond à notre définition de la complexité :  $T(n) = 1b + T(n-1)$  ( $P_1$ )

o Cas de récurrence :  $2 < i < n$

On suppose que  $P_i$  est vraie, et on veut prouver  $P_{i+1}$  :

$$T(n) = ib + T(n-i) \quad (P_i)$$

Or on a par définition de  $T$  :  $T(n-i) = b + T(n-i-1)$

On remplace  $T(n-i)$  dans  $P_i$  :

$$T(n) = ib + (b + T(n-i-1))$$

$$T(n) = i + 1 b + T(n - (i + 1)) \quad (P_{i+1})$$

o On a prouvé  $P_i$ .

4. On peut donc maintenant se ramener à une expression de  $T(n)$  en posant  $i = n-1$  :

$$T(n) = (n-1)b + T(n - (n-1))$$

$$T(n) = (n-1)b + a$$

o Au final, la complexité temporelle est donc  $T(n) = O(n) + O(1) = O(n)$

**Remarque :** la complexité spatiale est identique, puisqu'elle repose sur la même définition récursive.

**c) Formulaire**

Les mêmes méthodes pourraient s'appliquer à toutes les relations de récurrence. Cependant, les traiter en détail sort du cadre de ce cours. Comme nous allons quand même avoir besoin de calculer les complexités de certains algorithmes, on se propose plutôt d'utiliser le tableau suivant, qui associe une complexité asymptotique à différentes formes de récurrence.

Dans ce tableau, on suppose que le cas de base est  $T(1) = O(1)$ , et que  $k \geq 0$ .

| Relation de récurrence                        | Complexité        |
|-----------------------------------------------|-------------------|
| $T(n) = T(n-1) + \Theta(n^k)$                 | $O(n^{k+1})$      |
| $T(n) = cT(n-1) + \Theta(n^k)$ pour $c > 1$   | $O(c^n)$          |
| $T(n) = cT(n/d) + \Theta(n^k)$ pour $c > d^k$ | $O(n^{\log_d c})$ |
| $T(n) = cT(n/d) + \Theta(n^k)$ pour $c < d^k$ | $O(n^k)$          |
| $T(n) = cT(n/d) + \Theta(n^k)$ pour $c = d^k$ | $O(n^k \log n)$   |

# Chapitre 3 Structures de données

## 1. Introduction

Dans ce chapitre, nous passons en revue les structures de données élémentaires, à savoir les piles, les files ainsi que les arbres binaires et les arbres binaires de recherche. Nous décrivons ensuite les files de priorité et leur implémentation au moyen de tas.

## 2. Piles de données

La pile de données fait partie des structures de données avancées classiques de l'algorithmique. Nous allons dans cette section décrire ce type de façon théorique en nous reposant sur le concept de type abstrait, puis nous introduiront deux implémentations différentes à titre d'illustration.

### a) Présentation

Une pile est une structure de données très simple mais aussi très répandue. En anglais, on parle de stack, ou LIFO (pour Last In, First Out), qui Tous viennent du mode de fonctionnement de la pile. Une pile contient des informations homogènes, i.e. toutes de même type. Ce type peut être simple ou complexe. La particularité d'une pile est que seul le dernier élément inséré, qui est appelé le sommet, est accessible directement.

**Sommet de la pile :** dernier élément inséré dans la pile.

Quand un nouvel élément est inséré dans une pile, il est placé par-dessus l'ancien sommet, et en le recouvrant il devient lui-même le nouveau sommet. Cette opération est appelée l'empilement. L'opération inverse, consistant à supprimer un élément, est appelée le dépilement.

exemple : évolution d'une pile d'entiers dans laquelle on ajoute successivement les valeurs 6, 3 et 16, puis on supprime la valeur 16, puis on ajoute la valeur 87.

Le principe de pile est notamment utilisé :

- Dans les systèmes d'exploitation, pour gérer l'espace mémoire associé aux différents appels de fonction faits au sein d'un programme.
- Dans les compilateurs ou analyseurs syntaxiques, entre autres pour analyser des expressions bien parenthésées

### b) Type abstrait

- **Définition générale**

Avant de donner une implémentation des piles, nous introduisons la notion de type abstrait.

**Type abstrait :** définition axiomatique d'une structure de données.

On peut voir le type abstrait comme une spécification mathématique d'une structure de données, quelque chose de purement théorique. On se concentre seulement sur la description du comportement de la structure de données, et non pas sur la façon dont ce comportement va être réalisé.

**L'intérêt du type abstrait** est qu'il permet de donner une définition complètement indépendante de l'implémentation. La conséquence de cette propriété est qu'il est possible de proposer différentes implémentations différentes pour la même structure de données.

**Type concret :** résultat de l'implémentation d'un type abstrait.

Un type abstrait se compose d'un ensemble d'opérations et d'axiomes. Ils peuvent manipuler le type abstrait qu'on est en train de définir, ainsi que d'autres types prédéfinis.

- **Type abstrait Piles de données**

Nous donnons ci-dessous le type abstrait pour les piles de données. Chaque opération est listée avec son nom, sa description et son type.

## Opérations

- `cree_pile` : créer une pile vide  
`_`  $\rightarrow$  pile
- `est_pile_vide` : déterminer si la pile est vide  
pile  $\rightarrow$  booléen
- `sommet` : renvoyer le sommet de la pile  
pile  $\rightarrow$  element
- `empile` : rajouter un élément dans une pile  
pile  $\times$  element  $\rightarrow$  pile
- `depile` : retirer un élément d'une pile  
pile  $\rightarrow$  pile

L'opération `cree_pile` est capable de créer une nouvelle pile vide, sans prendre aucun paramètre d'entrée (d'où le signe `_`). Au contraire, `est_pile_vide` a besoin de recevoir une pile afin de tester si elle est vide ou pas, d'où la présence du type pile. Cette fonction renvoie soit vrai, soit faux, donc un booléen.

L'opération `sommet` reçoit une pile et renvoie son sommet, qui est un élément. L'opération `empile` a besoin de deux paramètres : l'élément à empiler, et la pile qui va le recevoir. Elle renvoie cette même pile augmentée de l'élément concerné. L'opération `depile` reçoit une pile, retire son sommet, et renvoie cette pile diminuée d'un élément.

Les axiomes sont les contraintes que les opérations doivent respecter :

### Axiomes

$\forall e, p \in \text{element} \times \text{pile}, p \text{ étant non-vide} :$

- ☐ `est_pile_vide` :
  1. `est_pile_vide(cree())` = vrai
  2. `est_pile_vide(empile(p,e))` = faux
- ☐ `sommet` :
  3. `sommet(cree())` = indéfini (i.e. opération interdite)
  4. `sommet(empile(p,e))` = e
- ☐ `depile` :
  5. `depile(cree())` = indéfini
  6. `depile(empile(p,e))` = p

### c) Implémentation par tableau

Pour implémenter une pile avec un tableau, on a besoin de connaître en permanence la taille de la pile, i.e. combien de cases du tableau sont occupées par des éléments de la pile. La meilleure solution est d'utiliser un type structuré incluant à la fois :

- Le tableau qui représente la pile ;
- L'entier qui représente sa taille.

### Structure de données

Pile=structure

taille : élément ;

Contenu : tableau[1..max] de élément

finstructure ;

**En exercice, implémentez les opérations.**

**d) Implémentation par liste chaînée**

On peut utiliser une liste simplement chaînée pour implémenter une pile, car on n'aura jamais besoin de parcourir la liste dans les deux sens. On considère que le sommet est situé au début de la liste, on n'aura qu'à faire des insertion/suppression de l'élément situé en tête de liste

**Structure de données** (voire Chapitre liste chaînée)

**En exercice, implémentez les opérations.**

**3. Files de données**

Comme pour la pile de données (section 2), nous allons décrire les files de données par le biais du type abstrait, avant d'en proposer différentes implémentations.

**a) Présentation**

À l'instar de la pile, la file de données est très répandue dans différents domaines de l'informatique. En anglais, on parle de queue, ou FIFO (pour First In First Out).

Comme la pile, la file est homogène : tous ses éléments contiennent une valeur du même type, qui peut être aussi bien simple qu'homogène. Comme la pile, on n'a accès qu'à un seul élément, mais cette fois il s'agit de celui qui a été inséré depuis le plus longtemps, alors qu'avec la pile il s'agissait de celui qui avait été inséré depuis le moins longtemps.

La seule donnée qui peut être enlevée de la file est la plus ancienne, on l'appelle la tête :

**Tête de file** : élément ayant été inséré depuis le plus longtemps.

Lors d'une insertion, le nouvel élément est placé après l'élément le plus récent, qui s'appelle la fin de la file :

**Fin de file** : élément ayant été inséré depuis le moins longtemps.

Dans une file, l'opération consistant à insérer un élément s'appelle l'enfilement, alors que l'opération inverse consistant à en retirer un est le défilement.

Le principe de file est notamment utilisé :

- Dans les systèmes d'exploitation, pour gérer les impressions (spouleurs d'impression), l'ordonnancement des processus, et plus généralement l'attente.
- En intelligence artificielle, dans les algorithmes d'exploration d'arbres de recherche, pour effectuer par exemple un parcours en largeur.

**b) Type abstrait**

Le type abstrait file de données se définit, comme pour la pile, en spécifiant d'abord un ensemble d'opérations, puis les axiomes qu'elles doivent respecter.

**Opérations**

- `cree_file` : créer une file vide  
 $\_ \rightarrow \text{file}$
- `est_vide` : déterminer si la file est vide  
 $\text{file} \rightarrow \text{booléen}$
- `tete` : renvoyer la tête de la file  
 $\text{file} \rightarrow \text{element}$
- `enfile` : rajouter un élément dans une file  
 $\text{file} \times \text{element} \rightarrow \text{file}$
- `defile` : retirer un élément d'une file  
 $\text{file} \rightarrow \text{file}$

L'opération `cree_file` sert à créer une file vide, comme `cree_pile` pour les piles de données. L'opération `est_vide` teste si la file est vide, i.e. si elle ne contient aucun élément.

L'opération tete renvoie un pointeur sur le premier élément de la file, on peut la rapprocher de sommet pour la pile. L'opération enfile insère un élément à la fin de la file, tandis que defile supprime la tête de la file. On peut les comparer aux opérations empile et depile définies pour les piles de données.

### Axiomes

$\forall e, f \in \text{element} \times \text{file}, f \text{ étant non-vide} :$

- est\_vide :
  1. est\_vide(cree()) = vrai
  2. est\_vide(enfile(f,e)) : faux
- tete :
  3. tete(cree()) = indéfini (i.e. opération interdite)
  4. tete(enfile(f,e)) =
- e si est\_vide(f)
- tete(f) si  $\neg \text{est\_vide}(f)$
- defile :
  5. defile(cree()) = indéfini
  6. defile(enfile(f,e)) =
- cree() si est\_vide(f)
- enfile(e,defile(f) si  $\neg \text{est\_vide}(f)$

L'axiome 1 stipule qu'une file qui vient d'être créée est forcément vide. L'axiome 2 dit qu'au contraire, une file quelconque dans laquelle on a enfilé un élément ne peut pas être vide. L'axiome 3 indique qu'il est indéfini de demander la tête d'une file qui est vide. L'axiome 4 dit que si la file n'est pas vide, on peut distinguer deux cas quand on en demande la tête. Si la file ne contient qu'un seul élément (1er cas listé), alors la tête correspond à cet élément. Sinon, la tête sera la même qu'on la demande avant ou après l'enfilement. Autrement dit, l'enfilement n'affecte pas la tête de la file (ce qui n'était pas le cas pour les piles).

L'axiome 5 interdit de défiler une file vide : en effet, si la file ne contient aucun élément, il est impossible d'en supprimer un. L'axiome 6 dit que si la file n'est pas vide, on doit distinguer deux cas. Soit la file ne contient qu'un seul élément, et alors le fait de la défiler produira une liste vide. Soit la file contient plus d'un élément, et dans ce cas-là on obtient une file non-vide, sans qu'on puisse en dire plus à son sujet.

### c) Implémentation simple par tableau

Comme pour les piles, on peut implémenter le type abstrait file de données en utilisant un tableau. Cependant, cette fois on peut distinguer deux implémentations différentes : l'une utilise le tableau comme on l'avait fait pour les piles, alors que l'autre adopte une approche circulaire.

### Structure de données

Pile=structure

taille : élément ;

Contenu : tableau[1..max] de élément

fstructure ;

**En exercice, implémentez les opérations.**

### d) Implémentation par liste chaînée

### Structure de données

Nous utiliserons une liste doublement chaînée comme structure de données dans, on aura aussi besoin de manipuler à la fois le début (pour défiler) et la fin (pour enfiler) de la liste.

Arbitrairement, on décide de placer la tête au début de la liste.

**Proposez la structure de données** (voire Chapitre liste chaînée)



## En exercice, implémentez les opérations.

### 4. Arbres

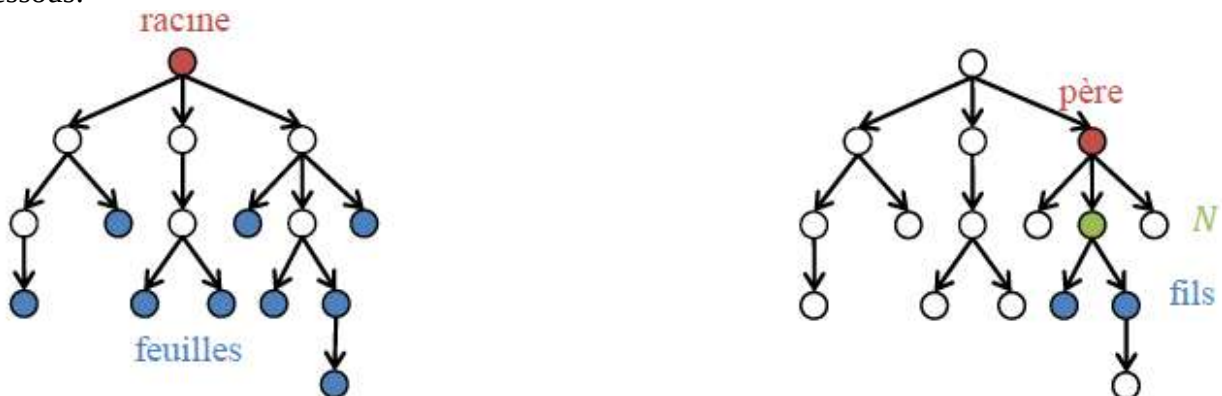
#### a) Généralités

Nous présentons dans cette section une structure de données un peu plus avancées que les piles et les files. Les arbres sont un concept lui aussi très utilisé dans de nombreux contextes de l'informatique, non seulement en tant que structure de données, mais aussi plus généralement comme outil de modélisation. Il existe de nombreuses variantes de la structure d'arbre, et nous allons nous concentrer ici aux arbres binaires.

Un arbre est une espèce particulière de graphe orienté :

**Arbre** : graphe acyclique orienté dans lequel chaque noeud n'a qu'un seul parent, sauf la racine unique qui n'en a pas du tout.

Le terme acyclique signifie que l'arbre ne contient aucun cycle. Un arbre ne contient qu'une seule racine (i.e. noeud sans parent) et tous les autres noeuds ont exactement un parent. Le résultat est une structure hiérarchique, telle que celles données en exemples dans la figure ci-dessous :



Chaque noeud contient une information appelée label (ou étiquette, ou clé). À cause des contraintes spécifiques aux arbres, un père peut avoir plusieurs fils, qui sont alors des frères.

**Frères** : ensemble de noeuds ayant le même parent.

Il existe aussi des noeuds qui n'ont aucun fils, et que l'on appelle les feuilles de l'arbre :

**Feuille** : noeud ne possédant aucun fils.

Un arbre est caractérisé par son arité :

**Arité d'un arbre** : nombre maximal de fils que peut avoir un noeud dans cet arbre.

En cas d'arité 2, on parle d'arbre binaire, puis d'arbre ternaire pour une arité 3, et plus généralement arbre  $n$ -aire pour une arité  $n$ .

#### Chemins

Par définition, il n'y a pas de cycle (chemin fermé) dans un arbre.

Les seuls chemins possibles sont descendants, i.e. ils partent d'un noeud et s'éloignent de la racine. Ceux reliant la racine à une feuille sont appelés branches :

En ce qui concerne les noeuds, on peut généraliser les concepts de parent et fils en définissant les ascendants et descendants :

**Ascendants (ou ancêtres) d'un noeud** : ensemble des noeuds constituant le chemin qui va de la racine au noeud considéré.

Par définition, la racine est donc un ascendant de tous les noeuds contenus dans l'arbre.

**Descendants d'un noeud** : ensemble des noeuds dont le noeud considéré est l'ascendant.

On peut également caractériser de façon numérique la position d'un noeud dans l'arbre :

**Profondeur d'un noeud** : longueur du chemin allant de la racine jusqu'au noeud considéré.

## Propriétés

Un arbre peut lui aussi être caractérisé par différentes propriétés.

**Hauteur d'un arbre :** profondeur de son noeud le plus profond.

La hauteur décrit la taille de l'arbre. D'autres propriétés décrivent sa forme :

**Arbre complet :** arbre  $n$ -naire de hauteur  $h$  dans lequel tout noeud de profondeur strictement inférieure à  $h$  possède exactement  $n$  fils.

## b) Arbre binaire

Nous nous intéressons ici aux arbres binaires, qui sont une forme simple d'arbre possédant de nombreuses applications en informatique.

**Arbre binaire :** arbre dans lequel un noeud peut avoir 0, 1 ou 2 fils.

Un arbre binaire peut être décomposé en trois parties :

- Sa racine ;
- Son sous-arbre gauche ;
- Son sous-arbre droit.

De même, chacun des sous-arbres peut être lui aussi décomposé de la même manière. Le type abstrait arbre binaire est basé sur cette décomposition récursive.

## c) Type abstrait

Comme pour les piles et les files de données, le type abstrait proposé ici se compose d'un ensemble d'opérations et d'un ensemble de contraintes qui leur sont appliquées.

### Opérations

- `cree_arbre` : créer un arbre vide  
 $\_ \rightarrow \text{arbre}$
- `est_vide` : déterminer si l'arbre est vide  
 $\text{arbre} \rightarrow \text{booléen}$
- `racine` : renvoyer la valeur située à la racine de l'arbre  
 $\text{arbre} \rightarrow \text{valeur}$
- `fils_gauche` : renvoyer le sous-arbre gauche d'un noeud  
 $\text{arbre} \rightarrow \text{arbre}$
- `fils_droit` : renvoyer le sous-arbre droit d'un noeud  
 $\text{arbre} \rightarrow \text{arbre}$
- `enracine` : créer un arbre à partir d'une valeur et de deux sous-arbres  
 $\text{valeur} \times \text{arbre} \times \text{arbre} \rightarrow \text{arbre}$

Les opérations permettant de créer un arbre et de tester s'il est vide sont similaires à celles déjà définies pour les piles et les files.

Les opérations `racine`, `fils_gauche` et `fils_droit` sont des opérations d'accès, elles permettent de récupérer les trois constituants de l'arbre et d'y naviguer. À noter que `racine` renvoie la valeur associée à la racine de l'arbre (i.e. son label ou sa clé), alors que `fils_gauche` et `fils_droit` renvoient ses deux sous-arbres, qui sont eux-mêmes du type arbre. On voit déjà qu'on aura une structure de données récursive, comme c'était déjà le cas avec les listes chaînées.

L'opération `enracine` permet de construire un arbre : elle crée un nouveau noeud en associant une racine et deux sous-arbres. Elle renvoie le nouvel arbre obtenu.

### Axiomes

- $\forall v, a1, a2 \in \text{valeur} \times \text{arbre} \times \text{arbre} :$
- `est_vide` :
  - `est_vide(cree()) = vrai`
  - `est_vide(enracine(v,a1,a2)) : faux`

- racine :
  - $\text{racine}(\text{cree}()) = \text{indéfini}$  (i.e. opération interdite)
  - $\text{racine}(\text{enracine}(v, a1, a2)) = n$
- fils\_droit :
  - $\text{fils\_droit}(\text{cree}()) = \text{indéfini}$
  - $\text{fils\_droit}(\text{enracine}(v, a1, a2)) = a2$
- fils\_gauche :
  - $\text{fils\_gauche}(\text{cree}()) = \text{indéfini}$
  - $\text{fils\_gauche}(\text{enracine}(v, a1, a2)) = a1$

D'après l'axiome 1, un arbre qui vient d'être créé est forcément vide. Au contraire, l'axiome 2 stipule qu'un arbre auquel on vient d'associer un noeud et deux sous-arbre ne peut pas être vide.

L'axiome 3 indique qu'un arbre vide ne contient aucun noeud, et qu'on ne peut donc pas demander sa racine. Au contraire, l'axiome 4 dit que racine renverra le dernier noeud inséré dans l'arbre, qui se trouve donc être sa racine.

Les axiomes 5 et 7 indiquent qu'un arbre vide ne contient pas de sous-arbre droit ni gauche.

Les axiomes 6 et 8, au contraire, stipulent qu'un arbre non-vide contient forcément des sousarbres droit et gauche, ceux-ci pouvant éventuellement être eux-mêmes vides.

#### **d) Implémentation par pointeur**

La définition d'une structure de données spécifique est nécessaire pour l'implémentation de l'arbre.

##### **Structure de données**

Dans notre structure de données, chaque noeud doit contenir les informations suivantes :

- Une valeur représentant son étiquette ;
- Un pointeur vers son fils gauche ;
- Un pointeur vers son fils droit.

Nœud=structure

Type Arbre = ^Nœud

Type Nœud = Structure

Info : TypeElement

FGauche : Arbre ;

Fdroit : Arbre ;

finStructure

**En exercice, implémentez les opérations.**

# Chapitre 4 Les fichiers séquentiels

## 1- Introduction

Un fichier est une représentation de données stockées sur un support persistant :

**Fichier** : ensemble de données associées à un nom et un emplacement sur un support de stockage (disque dur, clé USB, disque optique, etc.). le langage offre deux façons d'accéder aux fichiers :

- **Fonctions de bas niveau** :
  - Les fonctions correspondent à des routines du système d'exploitation (OS).
  - Ce type d'accès est donc très dépendant de l'OS utilisé, et par conséquent les programmes utilisant ces fonctions ne sont pas portables.
- **Fonctions de haut niveau** :
  - Fonctions indépendantes de l'OS, et donc programmes portables.

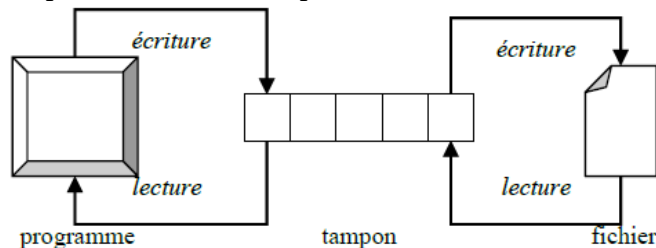
Dans ce cours nous n'utiliserons que les fonctions de haut niveau. Celles de bas niveau seront vues en cours de systèmes d'exploitation.

## 2- Structure FILE

L'accès aux fichiers est bufférisé, c'est-à-dire qu'on utilise un tampon (buffer) lors des écritures et des lectures.

**Mémoire tampon** : zone de mémoire utilisée pour y manipuler des données de façon temporaire, avant de les placer ailleurs. Quand on veut écrire dans un fichier, on écrit d'abord dans une zone spécifique de la mémoire appelée tampon. Quand cette zone est pleine, le système écrit son contenu dans le fichier, puis on recommence à remplir le tampon. Cette méthode permet de limiter le nombre d'accès en écriture dans le fichier, car ces accès sont relativement coûteux en termes de ressources (surtout en temps).

De même, lorsqu'on demande une lecture, les données lues dans le fichier sont copiées dans un tampon. On ne lit donc pas directement dans le fichier, mais dans la mémoire tampon.



La manipulation des fichiers n'est pas directe, on utilise pour cela une représentation sous la forme d'une variable de type structure FILE, qui est défini dans stdio.h.

La structure FILE contient différentes informations concernant le fichier, en particulier :

- ☐ Un pointeur vers l'adresse de début du tampon ;
- ☐ Un pointeur vers la position courante dans le tampon ;
- ☐ La taille du tampon.

On n'a jamais besoin d'initialiser manuellement une variable de type FILE : ce sont les fonctions relatives à la gestion des fichiers qui s'en chargent. Lorsqu'on ouvre un fichier, on récupère un *pointeur* vers une variable de type FILE, qui est utilisée pour lire ou écrire dans le fichier.

### 3- Ouverture/fermeture

Avant de pouvoir lire ou écrire dans un fichier, il est nécessaire de l'ouvrir. C'est cette ouverture qui permet d'initialiser une variable FILE pour représenter le fichier. L'ouverture est réalisée grâce à la fonction `fopen` dont l'en-tête est :

**`FILE* fopen(char *nom_fichier, char *mode);`**

La fonction renvoie un pointeur vers une structure FILE, ou bien NULL en cas d'erreur. Le paramètre `nom_fichier` est une chaîne de caractères contenant le chemin du fichier à ouvrir, i.e. son emplacement et son nom. Le paramètre `mode` est également une chaîne de caractères, qui indique comment le fichier doit être ouvert. Il existe de nombreux modes, mais dans le cadre de ce cours, nous utiliserons les plus simples :

- "r" : lecture.
  - o Renvoie NULL si le fichier n'existe pas.
- "w" : écriture.
  - o Si le fichier existe : son contenu est réinitialisé.
  - o Sinon : il est créé
- "a" : ajout de données à la fin du fichier.
  - o Fichier créé s'il n'existe pas déjà.
  - o Sinon, permet d'écrire de nouvelles données après les données existantes.

*exemples* : ouverture en lecture d'un fichier appelé `mon_fichier.txt` :

```
FILE *fp;
if((fp = fopen("mon_fichier.txt", "r")) == NULL)
{ //traitement en cas d'erreur
  ...
}
else
{ // traitement normal
  ...
}
```

La *fermeture* d'un fichier est l'opération réciproque de l'ouverture. Elle permet de notamment de libérer les ressources que le système avait réservé pour manipuler le fichier lors de l'ouverture, comme par exemple la variable de type FILE associée. En cas d'écriture, la fermeture permet aussi d'écrire dans le fichier les données encore présentes dans le buffer.

Elle est réalisée par la fonction `fclose`, dont l'en-tête est :

**`int fclose(FILE *fichier);`**

Le paramètre `fichier` est un pointeur vers une structure FILE représentant le fichier à fermer. Bien sûr, ce fichier doit avoir préalablement été ouvert par `fopen`. Si la fonction réussit, elle renvoie la valeur 0. Si elle échoue, elle renvoie la constante EOF (End of File). Cette constante est fréquemment utilisée dans les fonctions manipulant des fichiers, pour indiquer qu'un comportement anormal s'est produit.

*exemple* : fermeture du fichier précédemment ouvert

```
if((fclose(fp)) != EOF)
{ //traitement en cas d'erreur
  ...
}
else
{ // traitement normal
  ...
}
```

### 4- Lecture/écriture non-formatées en mode caractère

On distingue deux sortes d'accès non-formaté :

- Mode *caractère* : on lit/écrit un seul caractère à la fois ;

□ Mode *chaîne* : plusieurs caractères (une chaîne de caractère) simultanément.

Commençons par le premier mode, qui est aussi le plus simple.

La lecture est réalisée grâce à la fonction `fgetc` :

```
int fgetc(FILE *fichier);
```

Le paramètre est un pointeur vers une structure `FILE` représentant un fichier déjà ouvert.

La fonction renvoie le caractère lu, ou bien la constante `EOF` en cas d'erreur.

La valeur lue correspond à un `unsigned char`. On peut donc se demander pourquoi le type de retour de la fonction est `int`. Il se trouve que la constante `EOF` a généralement la valeur `-1`. Cette valeur négative ne peut pas être codée par un `unsigned char`, d'où l'utilisation d'un `int`.

*exemple* : on affiche le contenu d'un fichier déjà ouvert, et représenté par le pointeur `fp`

```
int c;
```

```
while((c=fgetc(fp)) != EOF)
```

```
    putchar(c);
```

Remarquez que l'on utilise dans la condition du `while` la même écriture compacte que précédemment dans le `if` du `malloc` (section 11.3). Si on voulait détailler le code source, on pourrait écrire de façon équivalente :

```
int c = fgetc(fp);
```

```
while(c!=EOF)
```

```
    { putchar(c);
```

```
      c = fgetc(fp);
```

```
    }
```

Lorsqu'arrive à la fin du fichier, le programme tente de lire la valeur suivante, ce qui est impossible. Cela déclenche donc une erreur, et `fgetc` renvoie `EOF`. Le problème est que la valeur `EOF` peut aussi être renvoyée à cause d'une autre erreur, par exemple si l'accès au fichier n'est plus possible. Il est donc nécessaire de pouvoir distinguer ces deux cas.

Pour être sûr que l'erreur correspond bien à une fin de fichier, on utilisera la fonction `feof` :

```
int feof(FILE *fichier);
```

Elle renvoie une valeur non-nulle si la position courante correspond à la fin du fichier représenté par le pointeur. Sinon, elle renvoie la valeur 0.

*exemple* : révision de l'exemple précédent en utilisant `feof` :

```
int c;
```

```
while((c=fgetc(fp)) != EOF)
```

```
    putchar(c);
```

```
    if(!feof(fp))
```

```
        { // traitement de l'erreur
```

```
        ...
```

```
    }
```

L'écriture en mode caractère est réalisée grâce à la fonction `fputc` :

```
int fputc(int caractere, FILE *fichier);
```

La fonction prend un `int` en paramètre, mais il est converti en `unsigned char`. Ce paramètre est de type `int` uniquement par souci de consistance avec `fgetc`. En cas de succès, la fonction renvoie la valeur qui a été écrite. Sinon, elle renvoie `EOF`.

*exemple* : on écrit dans un fichier

```
int c=5;
```

```
if((fputc(c,fp)) == EOF)
```

```
    { // traitement de l'erreur
```

```
    ...
```

```
    }
```

Encore une fois, remarquez l'utilisation de l'écriture compacte dans la condition du if.

## 5- Lecture/écriture non-formatées en mode chaîne

Au lieu de lire un seul caractère à la fois, on peut manipuler des chaînes de caractères. La lecture d'une chaîne de caractères est effectuée grâce à la fonction `fgets` dont l'en-tête est :

**`char* fgets(char *chaine, int nombre, FILE *fichier);`**

Le pointeur chaîne désigne le tampon, c'est-à-dire l'emplacement en mémoire où la chaîne lue doit être stockée (par exemple : un tableau de caractères). L'entier nombre indique le nombre maximal de caractères à lire. Le pointeur fichier désigne le fichier à lire (qui a été préalablement ouvert, bien entendu).

La fonction renvoie un pointeur vers le tampon, ou bien NULL en cas d'erreur. À noter que la fonction renvoie également NULL en cas de fin de fichier : comme pour `fgetc`, la fonction `feof` permet de distinguer une erreur d'une fin de fichier normale.

La fonction tente de lire nombre-1 caractères dans le fichier. Elle rajoute automatiquement le caractère '\0' à la fin de la chaîne. La lecture est interrompue si la fonction rencontre le caractère '\n' (le '\n' est quand même recopié dans le tampon).

*exemple :*

Soit un fichier contenant les lignes suivantes :

université

de

dschang

Supposons qu'on utilise l'instruction suivante :

**`char tampon[12];`**

**`while(fgets(tampon,12,fp) != NULL)`**

**`printf("%s",tampon);`**

alors la première chaîne obtenue sera :

{'u','n','i','v','e','r','s','i','t','é','\n','\0'}

la deuxième sera :

{'d','e','\n','\0'}

et la troisième sera :

{'d','s','c','h','a','n','g','\0'}

L'écriture d'une chaîne est effectuée grâce à la fonction `fputs` :

**`int fputs(char *chaine, FILE *fichier);`**

Le pointeur chaîne désigne la chaîne de caractères à écrire dans le fichier. La fonction ne recopie pas le caractère '\0' terminal. La fonction renvoie une valeur non-négative en cas de succès, ou bien EOF en cas d'erreur.

*exemple :* pour créer un fichier correspondant à l'exemple de `fgets`

`if(fputs("université\nde\ndschang\n", fp) == EOF)`

`// traiter l'erreur`

`if(fputs("université\nde\ndschang\n", fp) == EOF)`

`// traiter l'erreur`

# Chapitre 5 Tris

## 1. Introduction

Dans ce chapitre, nous présentons trois algorithmes de tri parmi les plus efficaces lorsqu'aucune hypothèse particulière n'est faite sur la nature ou la structure des clés des éléments à trier. Le tri rapide, le tri fusion et le tri par tas. Pour chaque technique de tri, nous présentons son principe et son implémentation.

## 2. Le tri rapide

### a) Principe

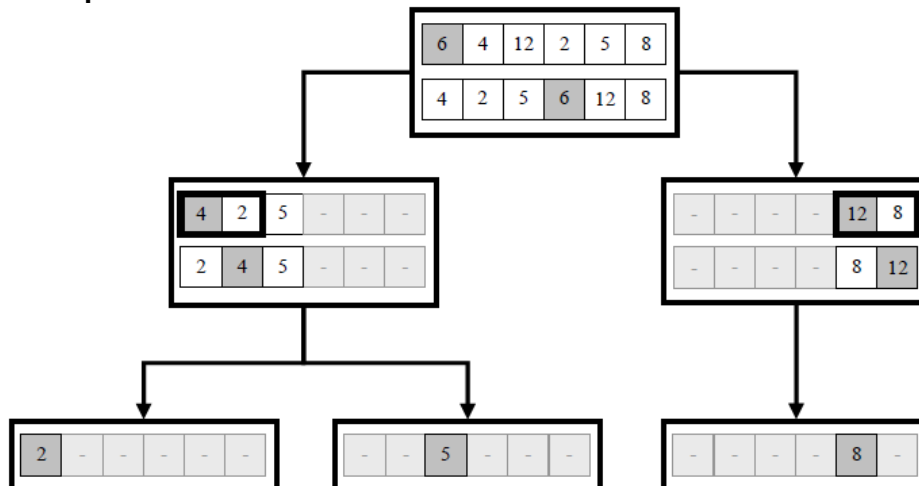
Le tri rapide est également appelé : *tri de Hoare*, *tri par segmentation*, *tri des bijoutiers*...

L'algorithme est le suivant :

- On choisit un élément du tableau qui servira de *pivot*.
- On effectue des échanges de valeurs dans le tableau, de manière à ce que :
  - Les valeurs *inférieures* au pivot soient placées à sa *gauche*.
  - Les valeurs *supérieures* au pivot soient placées à sa *droite*.
- On applique récursivement ce traitement sur les deux parties du tableau (i.e. à *gauche* et à *droite* du pivot).

**Remarque :** on peut remarquer que le choix du pivot est *essentiel*. L'idéal est un pivot qui partage le tableau en deux parties de tailles égales. Mais déterminer un tel pivot coûte trop de temps, c'est pourquoi il est en général choisi de façon arbitraire. Ici par exemple, on prend la première case du tableau. Entre ces deux extrêmes, des méthodes d'approximation peu coûteuses existent pour faire un choix de compromis.

### Exemple



Dans la figure ci-dessus, la valeur indiquée en gris correspond au pivot sélectionné pour diviser le tableau en deux.

### b) Implémentation

Soit `tri_rapide(tab[] : Vecteur, d : entier, f : entier)` la fonction récursive qui trie le tableau d'entiers `tab` de taille `N` en utilisant le principe du tri rapide. Les paramètres `d` et `f` marquent respectivement le début et la fin du sous-tableau en cours de traitement. Ces deux paramètres valent donc respectivement 0 et `N-1` lors du premier appel.

Constante `Max=1000` ;



Type vecteur=tableau[1..Max] de entier ;

fonction tri\_rapide(tab : Vecteur, d :Entier, f :Entier)

```
1 {   var taille: Entier;
2     taille=f-d+1;
3     var i,j,temp: Entier;
4     si (taille>1) alors
5     {       j=d;
6             pour (I allant de d+1 à f par pas de 1)
7             {       si (tab[i]<tab[d]) alors
8                     {       j=j+1;
9                             temp=tab[j];
10                            tab[j]=tab[i];
11                            tab[i]=temp;
12                        }
13                    }
14                temp=tab[d];
15                tab[d]=tab[j];
16                tab[j]=temp;
17                si (j>d) alors
18                    tri_rapide(tab,d,j-1);
19                si (j<f) alors
20                    tri_rapide(tab,j+1,f);
21            }
22    }
```

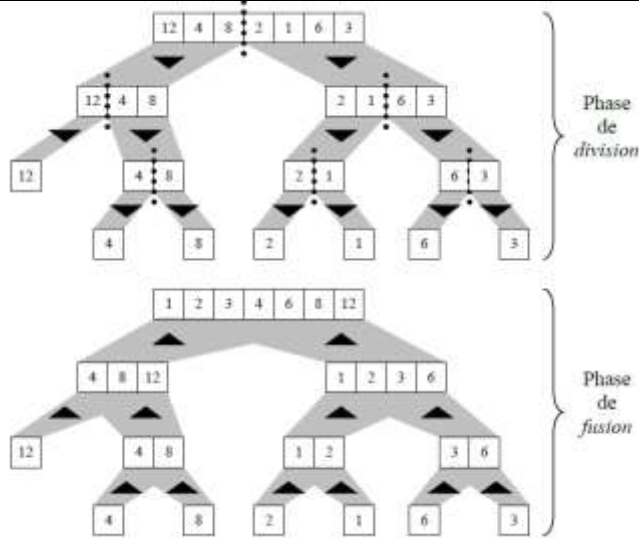
### 3. Le tri fusion

#### a) Principe

Le tri fusion fonctionne sur le principe diviser-pour-régner : plusieurs petits problèmes sont plus faciles à résoudre qu'un seul gros problème.

Le tri se déroule en trois étapes :

1. *Division* : le tableau est coupé en deux (à peu près) en son milieu ;
2. *Tri* : chaque moitié est triée séparément récursivement ;
3. *Fusion* : les deux moitiés triées sont fusionnées pour obtenir une version triée du tableau initial.



## b) Implémentation

Chaque phase de l'algorithme est implémentée dans une fonction différente.

La fonction `division(tab : Vecteur, tab1 : Vecteur, taille1 Entier, tab2 : Vecteur, taille2 :Entier)` divise le tableau d'entiers `tab` en deux parties stockées dans les tableaux `tab1` et `tab2`.

Les tailles de `tab1` et `tab2` sont respectivement `taille1` et `taille2`, et elles sont passées en paramètres (i.e. elles sont déjà calculées).

Fonction `division(tab : Vecteur, tab1 : Vecteur, taille1 Entier, tab2 : Vecteur, taille2 :Entier)`

```
{i:Entier;
1 pour (I allant de 0 à taille1 par pas de 1)
2   tab1[i]=tab[i];
3   pour (i allant de 0 à taille2 par pas de 1)
4     tab2[i]=tab[i+taille1];
   Finpour
 Finpour
}
```

Fonction `fusion(tab :Vecteur, tab1 :Vecteur, taille1 : Enteir,tab2:Vecteur, taille2:Entier)`

```
1 {i,j,k,m:Entier;
2   I=0,j=0,k=0;
3   tant que (i<taille1 && j<taille2)faire
3   {   si (tab1[i] < tab2[j]) alors
4       {   tab[k]=tab1[i];
5           i++;
        }
        sinon
6       {   tab[k]=tab2[j];
7           j++;
        }
8       k=k+1;;
    }
```

```

9 si (i!=taille1) alors
10 pour (m allant de i à taille1 par pas de 1)faire
11 {   tab[k]=tab1[m];
12     k++;
    }
    sinon
13     pour (m allant de j à taille2 par pas de 1) faire
14     {   tab[k]=tab2[m];
15         k++;
    }
}

```

La fonction tri\_fusion(tab :Vecteur, taille :Entier) trie récursivement un tableau tab de taille taille, grâce aux fonctions division et fusion.

```

fonction tri_fusion(tab :Vecteur, taille :Entier)
1 {   taille1 :Entier ;
2     taille1=taille/2;
3     taille2 :Entier ;
4     taille2=taille-taille/2;
5 tab1 :Vecteur1, tab2 :Vecteur2;
6 si (taille>1) alors
7 {   division(tab, tab1, taille1, tab2, taille2);
8     tri_fusion(tab2,taille2);
9     tri_fusion(tab1,taille1);
10    fusion(tab, tab1, taille1, tab2, taille2);
    }
}

```

#### 4. Tri par tas : Heapsort

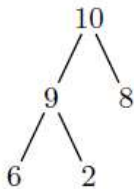
##### Définitions

- **Arbre binaire parfait** : tous les niveaux de l'arbre sont complets sauf le dernier que l'on remplit de gauche à droite.
- **Arbre partiellement ordonné** : la valeur d'un fils est supérieure ou égale à la valeur du père.
- **Tas** : arbre binaire et partiellement ordonné.

**Autres Définition** : Un tas est un arbre binaire dont tous les niveaux sont complètement remplis sauf éventuellement le dernier. De plus, toutes les feuilles doivent être le plus à gauche possible.

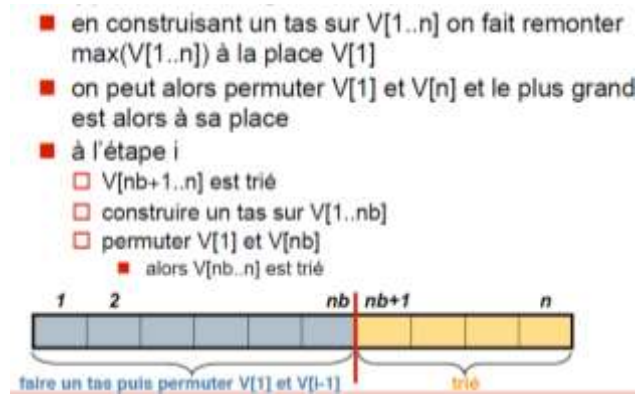
On peut le représenter par un tableau A et un entier taille tels que les éléments du tas soient dans A[1 . . . taille].

Un tas max est un tas tel que la racine de tout sous-arbre est plus grande que les éléments du sous-arbre.



Tas max représenté par [10; 9; 8; 6; 2]

## a) Principe



## b) Implémentation

```

procédure Tri-par-tas(T, n) ;
    p := 0; // création d'un tas vide
    pour k de 1 à n faire
        Insérer(T(k), T) ;
    finpour
    pour k de 1 à n faire
        extrairemin(T) ;
    finpour
finproc

```

Avant la  $k^{\text{ième}}$  insertion, le tas est constitué des  $k - 1$  premiers éléments de T et les éléments non encore insérés occupent les  $n - k$  positions suivantes. Après la  $k^{\text{ième}}$  suppression, le tas occupe les  $n - k$  premières positions du tableau et les éléments occupant les  $k$  dernières positions sont rangés par clé décroissante au sens large. En effet, l'échange des contenus de la racine et de la dernière feuille du tas, réalisé au début de  $\text{extrairemin}(T)$ , permet de récupérer l'élément supprimé lors de la  $k^{\text{ième}}$  suppression dans la position  $n - k + 1$  du tableau.

Pour l'insertion d'une clé  $x$ , on crée d'abord une feuille contenant cet élément (et qui est donc ajoutée au dernier niveau du tas) ; ensuite on compare son contenu à celui de son père; s'il est plus petit, on l'échange, et on continue en progressant vers la racine du tas.

```

procédure insérertas(x : clé, a : tas)
q := créer-feuille(x, a) ;
    tantque <>racine(a) et contenu(père(q)) > contenu(q) faire
        échanger-contenu(q, père(q)) ;
        q := père(q)
    fintantque.
finproc

```

L'extraction de la plus petite clé se fait sur un canevas similaire :

```

fonction extrairemin(a : tas);
    extrairemin := contenu(racine(a)) ;
    échanger-contenu(racine(a), dernière-feuille(a)) ;
    supprimer-feuille(a) ;
    si est-arbrevide(a) est faux alors
        p := racine(a) ;
        tantque non est-feuille(p) faire

```

```
    f := fils-min(p; a) ;  
    si contenu(f) < contenu(p) alors  
        échanger-contenu(f; p)  
    sinon  
        exit ;  
    finsi  
    p := f  
    fintantque  
finsi.
```

On remplace le contenu de la racine par le contenu de la dernière feuille. Cette feuille est supprimée. Puis, on descend de la racine vers les feuilles pour mettre la clé à une place convenable si elle ne l'est pas, c'est à dire si elle est plus grande que l'une des clés des fils.

## 5. Complexité des tris

Ces trois algorithmes de tri ont une complexité moyenne de  $O(n \log n)$ . Dans le pire des cas, le tri rapide a pour complexité  $O(n^2)$ , alors que la complexité des deux autres tris est de  $O(n \log n)$ .